

Resumen Algoritmia FOD

Procedimientos comunes:

```
1  const valorAlto = 'ZZZ' || 999;
2
3  procedure leer(var det: archivoDetalle; var regd: detalle);
4  begin
5      if (not EOF(det)) then
6          read(det, regd)
7      else
8          regd.cod := valorAlto;
9  end;
```

```
1  procedure minimo(
2      var det1, det2, det3: archivoDetalle;
3      var r1, r2, r3, min: detalle
4  );
5  begin
6      if (r1.cod <= r2.cod) and (r1.cod <= r3.cod) then begin
7          min := r1;
8          leer(det1, r1);
9      end else if (r2.cod <= r1.cod) and (r2.cod <= r3.cod) then begin
10         min := r2;
11         leer(det2, r2);
12     end else begin
13         min := r3;
14         leer(det3, r3);
15     end;
16 end;
```

- **Procedimiento leer:** Es fundamental para los casos de **N registros** o **Merge**. Verifica si se llegó al fin del archivo; si es así, asigna una constante de "valor alto" (como 'ZZZ' o 999) a la clave del registro para que las comparaciones de orden sigan funcionando sin desbordar el ciclo.
- **Procedimiento minimo:** Compara las claves de los registros actuales de cada archivo detalle. Selecciona el menor, lo devuelve como el registro "mínimo" y avanza la lectura únicamente en el archivo del cual provino ese registro.

Procedimientos específicos

- **Maestro – Detalle, un solo registro:** Cada registro del detalle modifica a un solo registro del maestro y ambos están ordenados por el mismo criterio.

```
1  reset(mae);
2  reset(det);
3
4  while (not EOF(det)) do begin
5      read(det, regd);
6      read(mae, regm);
7      while (regm.cod <> regd.cod) do
8          read(mae, regm);
9      regm.stock := regm.stock - regd.cant_vendida;
10     seek(mae, filepos(mae) - 1);
11     write(mae, regm);
12 end;
13
14 close(mae);
15 close(det);
```

- **Maestro – Detalle, varios registros:** Se agrupan (totalizan) todos los registros del detalle que tienen la misma clave antes de actualizar el maestro una sola vez.

```
1  reset(mae);
2  reset(det);
3
4  read(mae, regm);
5  leer(det, regd);
6
7  while (regd.cod <> valoralto) do begin
8      aux := regd.cod;
9      total := 0;
10     while (aux = regd.cod) do begin
11         total := total + regd.cant_vendida;
12         leer(det, regd);
13     end;
14     while (regm.cod <> aux) do
15         read(mae, regm);
16     regm.stock := regm.stock - total;
17     seek(mae, filepos(mae) - 1);
18     write(mae, regm);
19     if (not EOF(mae)) then
20         read(mae, regm);
21 end;
22
23 close(mae);
24 close(det);
```

- **Merge:** Se utiliza un procedimiento mínimo para comparar los registros actuales de todos los archivos detalle y escribir el menor en el maestro.

```

1  reset(det1);
2  reset(det2);
3  rewrite(mae);
4
5  leer(det1, r1);
6  leer(det2, r2);
7
8  minimo(det1, det2, r1, r2, min);
9
10 while (min.cod <> valoralto) do begin
11     aux:= min;
12     total := 0;
13     while (min.codigo = aux.codigo) do begin
14         total := total + min.cant;
15         minimo (det1, det2, det3,
16             regd1, regd2, regd3, min);
17     end;
18     aux.cant := total;
19     write (mae, aux);
20 end;
21
22 close(det1);
23 close(det2);
24 close(mae);

```

- **Corte de control:** Permite presentar información organizada por niveles. Requiere que el archivo esté estrictamente ordenado por los campos de los niveles (ej. provincia, ciudad).

```

1  reset(arch);
2  leer(arch, reg);
3
4  while (reg.prov <> valoralto) do begin
5      provAct := reg.prov;
6      totalProv := 0;
7      writeln('Provincia: ', provAct);
8      while (provAct = reg.prov) do begin
9          ciuAct := reg.ciu;
10         totalCiu := 0;
11         writeln('  Ciudad: ', ciuAct);
12         while (provAct = reg.prov) and (ciuAct = reg.ciu) do begin
13             totalCiu := totalCiu + reg.monto;
14             leer(arch, reg);
15         end;
16         writeln('    Total ciudad: ', totalCiu);
17         totalProv := totalProv + totalCiu;
18     end;
19     writeln('Total provincia: ', totalProv);
20 end;
21
22 close(arch);

```

Procedimientos de lista invertida

```
1  procedure crearArchivo(var arch: archivo);
2      var
3          l: libro;
4      begin
5          rewrite(arch);
6
7          // cabecera
8          l.codigo := 0;
9          write(arch, l);
10
11         leerLibro(l);
12         while (l.codigo <> 0) do begin
13             write(arch, l);
14             leerLibro(l);
15         end;
16
17         close(arch);
18     end;
```

```
1  procedure alta(var arch: archivo; l: libro);
2      var
3          cabecera, libre: libro;
4          pos: integer;
5      begin
6          reset(arch);
7          read(arch, cabecera);
8          if (cabecera.codigo < 0) then begin
9              // hay espacio libre
10             pos := -cabecera.codigo;
11             seek(arch, pos);
12             read(arch, libre);
13             // actualizar cabecera
14             seek(arch, 0);
15             write(arch, libre);
16             // escribir nuevo libro
17             seek(arch, pos);
18             write(arch, l);
19         end else begin
20             // no hay espacio libre → agregar al final
21             seek(arch, filesize(arch));
22             write(arch, l);
23         end;
24         close(arch);
25     end;
```

```

1  procedure baja(var arch: archivo; cod: integer);
2      var
3          cabecera, l: libro;
4          pos: integer;
5          encontrado: boolean;
6      begin
7          reset(arch);
8          read(arch, cabecera);
9          encontrado := false;
10         while (not eof(arch)) and (not encontrado) do begin
11             pos := filepos(arch);
12             read(arch, l);
13             if (l.codigo = cod) then begin
14                 encontrado := true;
15                 // enlazar con lista invertida
16                 l.codigo := cabecera.codigo;
17                 seek(arch, pos);
18                 write(arch, l);
19                 // actualizar cabecera
20                 cabecera.codigo := -pos;
21                 seek(arch, 0);
22                 write(arch, cabecera);
23             end;
24         end;
25         close(arch);
26     end;

```